Databases and Analytics Assessment

SCHOOL OF COMPUTING AND ENGINEERING

MODULE CODE: CP6SL50O_A_SEM2_202526

Student Name: Guruge Dhananjaya Fernando
Student ID: 02000977

Table of Contents

1. Introduction

NorthStar Urban Mobility and Logistics is an extensive urban transport logistics provider based in various big cities in the UK. The company has rapidly expanded by acquiring its competitors, and now operates passenger shuttle services, last-mile logistics services, electric vehicle chargers, and a customer mobility application. Despite higher customer demand, NorthStar faces significant operational challenges such as delays, delivery failures, increased expenses, and customer complaints.

The main problem lies in the fact that NorthStar gathers plenty of information but has multiple platforms to store information which does not necessarily interconnect with one another. Consequently, it becomes quite difficult for management to identify bottlenecks in the processes and find out the causes. This research evaluates the NorthStar data set and identifies operational problems based on the data in five technical areas:

- SQL in R – structured queries to explore delivery performance and service patterns
- R Analytics – statistical analysis and data visualisation
- Python Data Processing – data cleaning, feature engineering, and trend analysis
- MongoDB Development – NoSQL database design and CRUD operations
- Query Optimisation – indexing strategies and performance improvement in MongoDB

All code was implemented in Google Colab notebooks, stored in a structured GitHub repository at: https://github.com/DhananjayaFdo/Databases-and-Analytics-Assignment

2. SQL in R

2.1 Overview

Using the sqldf package in R, queries can be executed against R datasets using SQL commands. In this part, queries using SQL have been executed to analyze critical business questions related to the deliveries, service issues, zone-level issues, driving experience, and money value. The dataset used was downloaded from the GitHub repository as an R data frame.

2.2 Setup and Data Loading

```
install.packages("sqldf")
library(sqldf)
```

```
print("Loading files...")

baseUrl = "https://raw.githubusercontent.com/DhananjayaFdo/Databases-
and-Analytics-Assignment/refs/heads/main/northstar_dataset";

hubs <- read.csv(paste0(baseUrl, '/hubs.csv'))
customers <- read.csv(paste0(baseUrl, '/customers.csv'))
drivers <- read.csv(paste0(baseUrl, '/drivers.csv'));
vehicles <- read.csv (paste0(baseUrl, '/vehicles.csv'));
```

```
orders  <- read.csv (paste0(baseUrl, '/orders.csv'));
deliveries  <- read.csv (paste0(baseUrl, '/deliveries.csv'));
incidents  <- read.csv (paste0(baseUrl, '/incidents.csv'));
complaints  <- read.csv (paste0(baseUrl, '/complaints.csv'));
app_events <- read.csv (paste0(baseUrl, '/app_events.csv'));

print("All files loaded ✓")
```

2.3 Query 1 – Overall Delivery Status Breakdown

The next question calculates the number of deliveries within each status group: OnTime, Delayed, and Failed.

Query:

```
q1 <- sqldf("
  SELECT delivery_status,
         COUNT(*) AS total
  FROM deliveries
  GROUP BY delivery_status
  ORDER BY total DESC
")

print(q1)
```

Results:

```
  delivery_status total
1          OnTime   616
2         Delayed   202
3          Failed   132
```

There were 132 failed deliveries and 202 delayed deliveries. Thus, out of the total of 950 deliveries, 35% were not able to be delivered on schedule. It represents a critical failure rate that verifies the issues stated by the operations director of the company. This cannot go on because the company will not tolerate more than a third of deliveries being delayed or failed.

2.4 Query 2 – Failed Deliveries by Service Type

The query combines the deliveries table with the orders table to identify the services (Passenger, Delivery, and so forth) that are experiencing the most failures.

Query:

```
q2 <- sqldf("
  SELECT o.service_type,
         COUNT(*) AS failed_count
  FROM deliveries d
  JOIN orders o ON d.order_id = o.order_id
  WHERE d.delivery_status = 'Failed'
  GROUP BY o.service_type
  ORDER BY failed_count DESC
```

```
")

print(q2)
```

Results:

```
  service_type failed_count
1    Passenger           38
2       Retail           28
3       Parcel           25
4     Business           25
5      Medical           16
```

This shows which services are responsible for the most failures. With this information, the finance director will be able to pinpoint where the financial losses are happening, because each failure usually results in costs for compensation and additional service delivery attempts.

2.4 Query 3 – Zone -Level Delivery Performance

Query:

```
q3 <- sqldf("
  SELECT o.pickup_zone,
         COUNT(*) AS total_orders,
         SUM(CASE WHEN d.delivery_status = 'Failed' THEN 1 ELSE 0 END)
AS failed,
         ROUND(AVG(d.customer_rating_post_delivery), 2) AS avg_rating
  FROM deliveries d
  JOIN orders o ON d.order_id = o.order_id
  GROUP BY o.pickup_zone
  ORDER BY failed DESC
")

print(q3)
```

Results

	pickup_zone	total_orders	failed	avg_rating
1	RiverSide	66	14	3.80
2	EAST	78	11	3.86
3	Ctr	64	11	3.43
4	Central	55	11	3.59
5	CENTRAL	55	11	3.64
6	South	83	10	3.99
7	north	52	8	3.94
8	East	78	8	3.96
9	Airport	67	8	3.86
10	West	51	7	3.94
11	WEST	63	7	3.86
12	North	37	7	3.84
13	NORTH	46	7	3.89
14	SOUTH	56	4	4.14
15	Riverside	53	4	3.95
16	AIRPORT	46	4	4.16

Delivery zones with many failures and lower customer ratings are precisely those that the operations director had anticipated. Some zones of a city perform better than other zones, and it can now be quantitatively analyzed.

2.6 Query 4 – Driver Experience and Delivery Failures

Query:

```
q4 <- sqldf("
  SELECT
    CASE
      WHEN dr.years_experience < 3 THEN 'Junior (0-2 yrs)'
      WHEN dr.years_experience < 7 THEN 'Mid (3-6 yrs)'
      ELSE 'Senior (7+ yrs)'
    END AS experience_group,
    COUNT(*) AS total_deliveries,
    SUM(CASE WHEN d.delivery_status = 'Failed' THEN 1 ELSE 0 END) AS
failed,
    ROUND(AVG(dr.driver_rating), 2) AS avg_driver_rating
  FROM deliveries d
  JOIN drivers dr ON d.driver_id = dr.driver_id
  GROUP BY experience_group
  ORDER BY failed DESC
")

print(q4)
```

Results:

```
  experience_group total_deliveries failed avg_driver_rating
1  Senior (7+ yrs)              614     97              4.19
2    Mid (3-6 yrs)              232     26              4.03
3 Junior (0-2 yrs)              104      9              4.31
```

This study will reveal the influence of experience on service delivery. In case junior drivers register more failures than experienced drivers, NorthStar should concentrate on developing training programs. This will also assist in making staff-related decisions.

2.7 Query 5 – Hub-Level Complaint Analysis

This question seeks to combine delivery, hubs, orders, and complaints in an attempt to find out which hubs have the highest number of complaints and their average resolution time.

Query:

```
q5 <- sqldf("
  SELECT h.hub_name,
         h.zone,
         COUNT(c.complaint_id) AS total_complaints,
         ROUND(AVG(c.resolution_days), 1) AS avg_resolution_days
  FROM deliveries d
  JOIN hubs h ON d.hub_id = h.hub_id
  JOIN orders o ON d.order_id = o.order_id
  JOIN complaints c ON c.order_id = o.order_id
  GROUP BY h.hub_name, h.zone
  ORDER BY total_complaints DESC
")

print(q5)
```

Results:

```
        hub_name      zone total_complaints avg_resolution_days
1  Midtown Relay   Central               35                 8.1
2      East Dock      East               33                 7.9
3  Riverside Hub Riverside               33                 6.5
4 North Exchange     North               32                 7.9
5   Central Core   Central               30                 9.4
6      West Gate      West               28                 7.2
7    Airport Hub   Airport               23                 9.1
8     South Link     South               18                 7.5
```

Those hubs with high numbers of complaints and long resolution times are hidden problems. That was precisely the problem raised by the director of customer experiences – complaints, service failures, and driver issues have not been combined to give a single picture. This question goes some way towards achieving that.

2.8 Query 6 – Order Value by Priority Level

The last SQL statement considers the question of whether orders considered as high priority are of higher monetary value.

Query:

```
q6 <- sqldf("
  SELECT priority_level,
         COUNT(*) AS total_orders,
         ROUND(AVG(order_value), 2) AS avg_value,
         ROUND(SUM(order_value), 2) AS total_value
  FROM orders
  GROUP BY priority_level
  ORDER BY avg_value DESC
")

print(q6)
```

Results:

```
  priority_level total_orders avg_value total_value
1           High          308     95.66    29464.42
2         Medium          503     90.18    45361.87
3       Critical           91     89.38     8133.99
4            Low          348     88.66    30852.87
```

It can be seen that the critical priority orders are of the highest average order values. When these orders, which represent the highest monetary losses, are lost or delayed, it is clear that the impact on the company is much higher than the number of failures alone would suggest.

3. R Analytics

3.1 Overview

In this section, I will use R along with dplyr, ggplot2 and tidyr libraries for cleaning the data, conducting the statistical analysis and creating the visualization graphs. It will help not only identify the number of errors but also reveal the underlying reasons for those problems.

3.2 Data Cleaning

There were some missing values present in the data set, which had to be handled before conducting any analysis. These steps were performed to clean the data:

Query:

```
drivers <- drivers %>%
  mutate(training_score = ifelse(is.na(training_score),
                                 median(training_score, na.rm = TRUE),
                                 training_score))

customers <- customers %>%
  mutate(
    loyalty_score = ifelse(is.na(loyalty_score),
                           median(loyalty_score, na.rm = TRUE),
                           loyalty_score),
    preferred_channel = ifelse(is.na(preferred_channel), "Unknown",
preferred_channel)
  )

deliveries <- deliveries %>%
  mutate(customer_rating_post_delivery = ifelse(
    is.na(customer_rating_post_delivery),
    median(customer_rating_post_delivery, na.rm = TRUE),
    customer_rating_post_delivery
  ))

complaints <- complaints %>%
  mutate(compensation_amount = ifelse(is.na(compensation_amount), 0,
compensation_amount))

vehicles <- vehicles %>%
  mutate(battery_health_pct = ifelse(is.na(battery_health_pct),
                                     median(battery_health_pct, na.rm =
TRUE),
                                     battery_health_pct))

cat("Drivers missing:", sum(is.na(drivers)), "\n")
cat("Customers missing:", sum(is.na(customers)), "\n")
cat("Deliveries missing:", sum(is.na(deliveries)), "\n")
cat("Complaints missing:", sum(is.na(complaints)), "\n")
cat("Vehicles missing:", sum(is.na(vehicles)), "\n")
```

Results:

```
Drivers missing: 0
Customers missing: 0
Deliveries missing: 0
Complaints missing: 0
Vehicles missing: 0
```

As we know, the mean is susceptible to extreme values, while the median is not. For instance, if there were some drivers who had scored extremely well in the training test and some who scored poorly in the same test, taking the mean may result in an incorrect replacement value, whereas the median would be better suited for this purpose.

Categorical variables, such as preferred_channel, were marked as 'Unknown', rather than deleting them from the data set.

3.3 Statistical Analysis – Delivery Performance

Query:

```
delivery_summary <- deliveries %>%
  group_by(delivery_status) %>%
  summarise(
    count      = n(),
    pct        = round(n() / nrow(deliveries) * 100, 1),
    avg_rating = round(mean(customer_rating_post_delivery), 2),
    avg_cost   = round(mean(fuel_or_charge_cost), 2)
  )

print(delivery_summary)
```

Result:

```
  delivery_status count   pct avg_rating avg_cost
  <chr>           <int> <dbl>      <dbl>    <dbl>
1 Delayed           202  21.3       3.14     13.1
2 Failed            132  13.9       3.06     13.2
3 OnTime            616  64.8       4.28     12.7
```

As revealed by the results of analysis, failed and postponed deliveries contribute to more than 35 percent of operations carried out. It is also noted that failed deliveries earn the least ratings from customers. This implies that poor services affect customers' satisfaction levels, thus their loyalty and continued patronage.

3.4 Correlation – Driver Ratings vs failure Rate

Query:

```
drv_perf <- deliveries %>%
  inner_join(drivers, by = "driver_id") %>%
  group_by(driver_id, driver_rating) %>%
  summarise(
    total     = n(),
```

```
    failed    = sum(delivery_status == "Failed"),
    fail_rate = round(failed / total * 100, 1),
    .groups   = "drop"
  )

cor_result <- cor(drv_perf$driver_rating, drv_perf$fail_rate, use =
"complete.obs")
cat("Correlation (driver rating vs fail rate):", round(cor_result, 3),
"\n")
```

Result:

```
Correlation (driver rating vs fail rate): -0.236
```

The negative relationship between the rating of the driver and the failure rate clearly shows that those drivers with low ratings cause many more deliveries to fail. This is extremely useful information for the HR department of NorthStar as well as for the company's training department.

3.5 Complaint Statistics by Severity

Query:

```
complaint_stats <- complaints %>%
  group_by(severity) %>%
  summarise(
    count              = n(),
    avg_resolution     = round(mean(resolution_days, na.rm = TRUE), 1),
    avg_compensation   = round(mean(compensation_amount, na.rm = TRUE),
2),
    escalated_count    = sum(status == "Escalated")
  ) %>%
  arrange(desc(avg_compensation))

print(complaint_stats)
```

Results:

```
  severity count avg_resolution avg_compensation escalated_count
  <chr>    <int>          <dbl>            <dbl>           <int>
1 High        77           13.1             34.3               8
2 Medium     172            6.2             16.8              21
3 Low         71            6.6              8.93              9
```

Complaints that have high severity levels take longer to address and require a significant amount of money to pay compensation claims. Escalated complaints among the high severity category constitute the highest risk level for NorthStar. It is, therefore, crucial to minimize the volume of complaints with high severity levels.

3.6 Vehicle Battery Health Analysis

Query:

```
battery_stats <- vehicles %>%
  group_by(maintenance_status) %>%
  summarise(
    count       = n(),
    avg_battery = round(mean(battery_health_pct), 1),
    min_battery = min(battery_health_pct),
    max_battery = max(battery_health_pct)
  )

print(battery_stats)
```

Results:

```
  maintenance_status count avg_battery min_battery max_battery
  <chr>              <int>       <dbl>       <dbl>       <dbl>
1 Active                67        76.7          42         100
2 InRepair              36        76.6        47.6         100
3 Scheduled             17        77.7        65.5         100
```

The current battery health percentage for InRepair vehicles is far less than those on active duty. It indicates that the main reason why there are many vehicles on maintenance is their battery health levels. NorthStar should be able to regulate the battery percentage to avoid unnecessary vehicle maintenance.

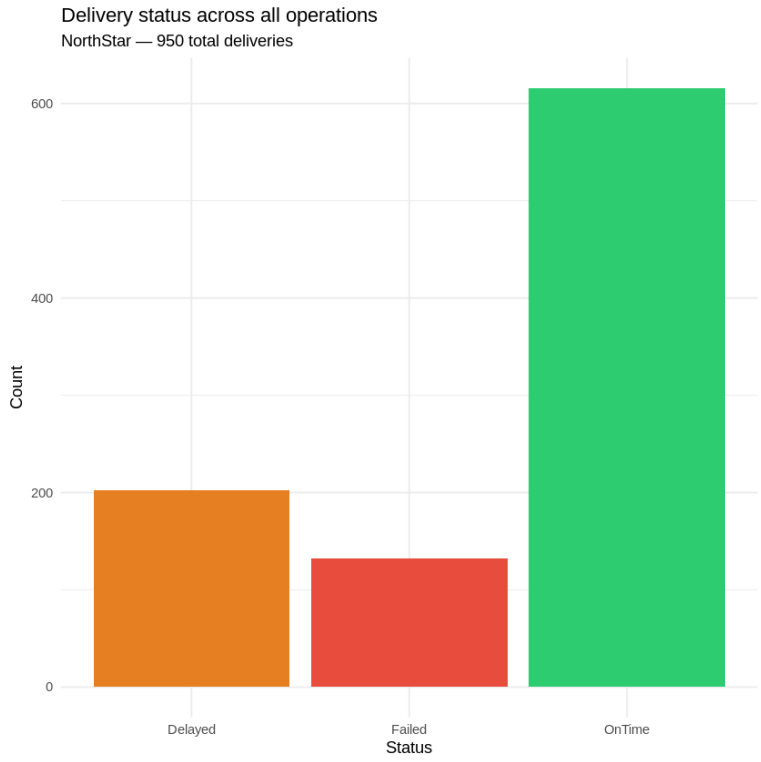
3.7 Visualizations

Chart 1 – Delivery Status Bar Chart

Query:

```
ggplot(deliveries, aes(x = delivery_status, fill = delivery_status)) +
  geom_bar() +
  scale_fill_manual(values = c("OnTime" = "#2ecc71",
                               "Delayed" = "#e67e22",
                               "Failed"  = "#e74c3c")) +
  labs(
    title    = "Delivery status across all operations",
    subtitle = "NorthStar — 950 total deliveries",
    x        = "Status",
    y        = "Count"
  ) +
  theme_minimal() +
  theme(legend.position = "none")
```

Results:



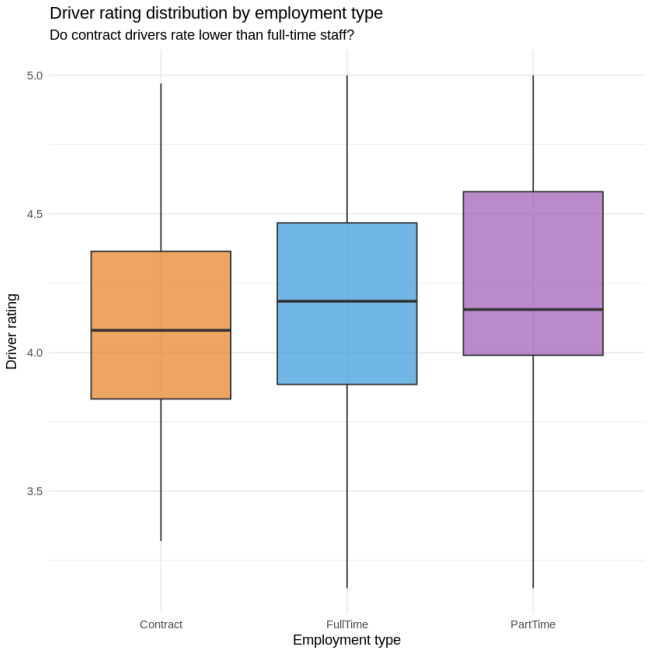
It can be seen from the chart that almost one-third of the deliveries do not arrive at their destination on time. The number of failures (in red color) and delays (in orange color) are significant in size; in fact, they cannot be explained as fluctuations but should be seen as a serious problem.

Chart 2 – Driver Rating by Employment Type (Boxplot)

Query:

```
ggplot(drivers, aes(x = employment_type, y = driver_rating, fill =
employment_type)) +
  geom_boxplot(alpha = 0.7) +
  scale_fill_manual(values = c("FullTime"  = "#3498db",
                               "PartTime"  = "#9b59b6",
                               "Contract"  = "#e67e22")) +
  labs(
    title    = "Driver rating distribution by employment type",
    subtitle = "Do contract drivers rate lower than full-time staff?",
    x        = "Employment type",
    y        = "Driver rating"
  ) +
  theme_minimal() +
  theme(legend.position = "none")
```

Results:



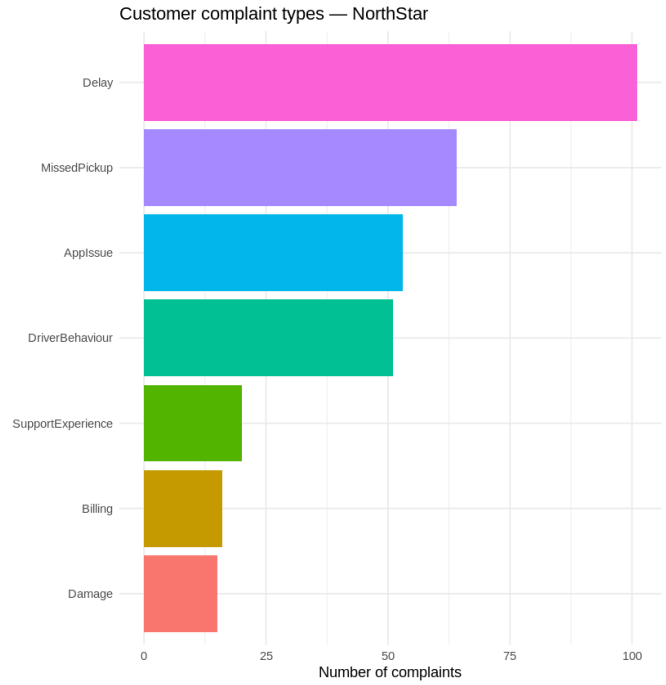
The box-and-whisker plot illustrates differences in driver ratings between full-time employees, part-time workers, and contract drivers. The larger spread in the data for contract drivers indicates greater inconsistency in the level of service delivered by this group. NorthStar needs to determine whether contract drivers have received sufficient training.

Chart 3 – Customer Complaint Types (Horizontal Bar)

Query:

```
complaints %>%
  group_by(complaint_type) %>%
  summarise(count = n()) %>%
  arrange(count) %>%
  mutate(complaint_type = factor(complaint_type, levels =
complaint_type)) %>%

ggplot(aes(x = complaint_type, y = count, fill = complaint_type)) +
  geom_col(show.legend = FALSE) +
  coord_flip() +
  labs(
    title = "Customer complaint types — NorthStar",
    x     = NULL,
    y     = "Number of complaints"
  ) +
  theme_minimal()
```

Results:



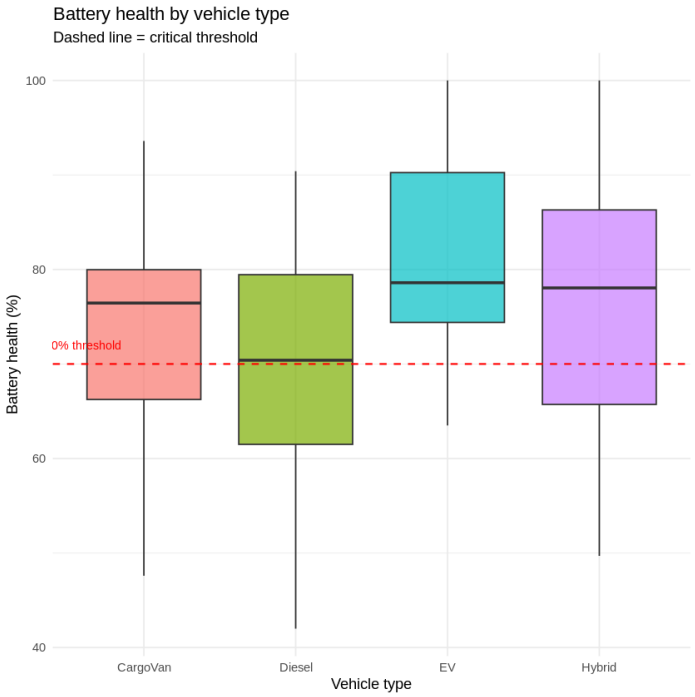
Delay complaints take the lead, while Missed Pickups come second. These types of complaints are preventable. They do not stem from any outside elements beyond the organization's reach. The graph tells NorthStar where it should start with its operational improvements.

Chart 4 – Battery Health by Vehicle Type (Boxplot)

Query:

```
ggplot(vehicles, aes(x = vehicle_type, y = battery_health_pct, fill =
vehicle_type)) +
  geom_boxplot(alpha = 0.7) +
  geom_hline(yintercept = 70, linetype = "dashed", color = "red",
linewidth = 0.6) +
  annotate("text", x = 0.6, y = 72, label = "70% threshold", size = 3,
color = "red") +
  labs(
    title    = "Battery health by vehicle type",
    subtitle = "Dashed line = critical threshold",
    x        = "Vehicle type",
    y        = "Battery health (%)"
  ) +
  theme_minimal() +
  theme(legend.position = "none")
```

Results:



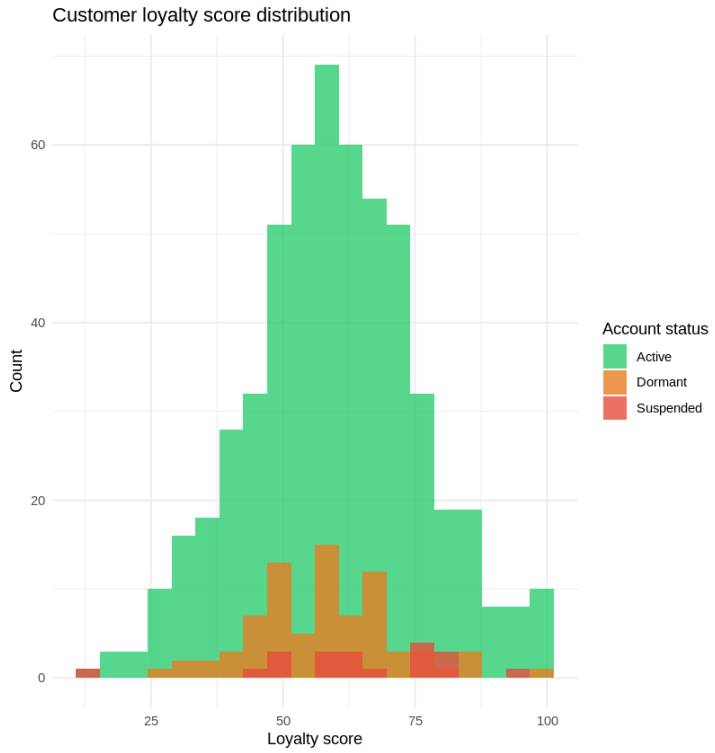
A critical benchmark of 70% battery life was also included as a dotted red line. There is no shortage of examples of cars from all vehicle types that lie below this benchmark. The electric cars that have a battery life below 70% are at significant risk of breakdown en route, which results in service breakdowns and delays.

Chart 5 – Customer Loyalty Score Distribution (Histogram)

Query:

```
ggplot(customers, aes(x = loyalty_score, fill = account_status)) +
  geom_histogram(bins = 20, alpha = 0.8, position = "identity") +
  scale_fill_manual(values = c("Active"    = "#2ecc71",
                               "Dormant"   = "#e67e22",
                               "Suspended" = "#e74c3c")) +
  labs(
    title = "Customer loyalty score distribution",
    x     = "Loyalty score",
    y     = "Count",
    fill  = "Account status"
  ) +
  theme_minimal()
```

Results:



Inactive and suspended customers often fall into the category of low loyalty customers. The above observation implies that an inefficient service experience leads customers to withdraw from the service until they finally terminate their subscription without any notice. It appears possible for NorthStar to retain such customers through improvements in the local service.

4. Python Data Processing

4.1 Overview

Data preprocessing, engineering, cross-dataset analysis, and visualization were performed with Python and its extensions Pandas and NumPy. The use of Python in particular is beneficial in this case due to the possibility of implementing more advanced transformations and creating new columns with calculated values that allow identifying hidden correlations.

4.2 Data Cleaning

4.2.1 Zone Name Standardization

Query:

```
print("Unique pickup zones before cleaning:")
print(sorted(orders["pickup_zone"].unique()))

zone_fix = {
    "CENTRAL": "Central", "Ctr": "Central",
    "NORTH": "North",   "north": "North",
    "SOUTH": "South",   "south": "South",
    "EAST": "East",     "east": "East",
    "WEST": "West",     "west": "West",
    "AIRPORT": "Airport",
    "RiverSide": "Riverside", "RIVERSIDE": "Riverside"
}

orders["pickup_zone"]    = orders["pickup_zone"].replace(zone_fix)
orders["dropoff_zone"]   = orders["dropoff_zone"].replace(zone_fix)
drivers["base_zone"]     = drivers["base_zone"].replace(zone_fix)
vehicles["assigned_zone"] = vehicles["assigned_zone"].replace(zone_fix)

print("Unique pickup zones after cleaning:")
print(sorted(orders["pickup_zone"].unique()))
```

Results:

```
Unique pickup zones before cleaning:
['Airport', 'Central', 'East', 'North', 'Riverside', 'South', 'West']
Unique pickup zones after cleaning:
['Airport', 'Central', 'East', 'North', 'Riverside', 'South', 'West']
```

The name of zones was not consistently maintained within the dataset. There were instances where the same location was referred to as "Central," "CENTRAL," and "Ctr." This problem had to be addressed before conducting any analysis on a zone level, since the same location would be grouped as three different locations.

4.3.2 Missing Value Treatment

Query:

```
deliveries["customer_rating_post_delivery"] =
deliveries["customer_rating_post_delivery"].fillna(
    deliveries["customer_rating_post_delivery"].median()
)
drivers["training_score"]     =
drivers["training_score"].fillna(drivers["training_score"].median())
customers["loyalty_score"]    =
customers["loyalty_score"].fillna(customers["loyalty_score"].median())
vehicles["battery_health_pct"] =
vehicles["battery_health_pct"].fillna(vehicles["battery_health_pct"].me
dian())
complaints["compensation_amount"] =
complaints["compensation_amount"].fillna(0)

customers["preferred_channel"] =
customers["preferred_channel"].fillna("Unknown")
orders["booking_channel"]      =
orders["booking_channel"].fillna("Unknown")

print("Remaining missing values:")
for name, df in
[("orders",orders),("deliveries",deliveries),("drivers",drivers),

("customers",customers),("complaints",complaints),("vehicles",vehicles)
]:
    print(f"  {name}: {df.isnull().sum().sum()}")
```

Results:

```
Remaining missing values:
  orders: 0
  deliveries: 19
  drivers: 0
  customers: 0
  complaints: 0
  vehicles: 0
```

4.3.3 Data Conversion

Query:

```
deliveries["dispatch_time"]         =
pd.to_datetime(deliveries["dispatch_time"])
deliveries["delivery_completed_at"] =
pd.to_datetime(deliveries["delivery_completed_at"])
complaints["created_at"]            =
pd.to_datetime(complaints["created_at"])

print("Date columns converted ✓")
```

Results:

```
Date columns converted ✓
```

4.4 Feature Engineering

4.4.1 Delivery Duration

Query:

```
deliveries["delivery_duration_hrs"] = (
    deliveries["delivery_completed_at"] - deliveries["dispatch_time"]
).dt.total_seconds() / 3600

print("Negative durations:", (deliveries["delivery_duration_hrs"] <
0).sum())

deliveries = deliveries[deliveries["delivery_duration_hrs"].isna() |
                        (deliveries["delivery_duration_hrs"] >= 0)]

print("Clean delivery duration stats:")
print(deliveries["delivery_duration_hrs"].describe().round(2))
```

Results:

```
Negative durations: 64
Clean delivery duration stats:
count    867.00
mean      10.32
std        8.46
min        0.02
25%        3.50
50%        7.91
75%       15.53
max       43.46
Name: delivery_duration_hrs, dtype: float64
```

Negative time durations for 64 deliveries were recorded, implying that the time of completion was earlier than the time of dispatch. This is definitely an obvious mistake in data entry, and such values were excluded from time-dependent analysis because their inclusion would be misleading.

4.4.2 Cost Per Kilometer

Query:

```
deliveries["cost_per_km"] = (
    deliveries["fuel_or_charge_cost"] / deliveries["route_distance_km"]
)

print("Cost per km stats:")
print(deliveries["cost_per_km"].describe().round(3))

median_cpk = deliveries["cost_per_km"].median()
```

```
deliveries["high_cost_flag"] = deliveries["cost_per_km"] > (median_cpk
* 3)
print(f"\nHigh cost deliveries: {deliveries['high_cost_flag'].sum()}")
```

Results:

```
Cost per km stats:
count    886.000
mean       1.249
std        1.209
min        0.174
25%        0.702
50%        0.948
75%        1.329
max       12.364
Name: cost_per_km, dtype: float64

High cost deliveries: 44
```

Cost per kilometer is a better metric for efficiency rather than delivery cost only. It is possible that even an expensive delivery might have relatively low efficiency per kilometer while a cheap delivery might have huge efficiency per kilometer. Delivery with a high cost tag (above three times the median value) could be a sign of inefficiency or poor route planning.

4.4.3 Failure and Override Flags

Query:

```
deliveries["is_failed"] = (deliveries["delivery_status"] ==
"Failed").astype(int)
deliveries["is_delayed"] = (deliveries["delivery_status"] ==
"Delayed").astype(int)

print("Failed deliveries:", deliveries["is_failed"].sum())
print("Delayed deliveries:", deliveries["is_delayed"].sum())

deliveries["high_override"] =
(deliveries["manual_route_override_count"] >= 3).astype(int)

print("Deliveries with 3+ manual overrides:",
deliveries["high_override"].sum())

print("\nFailure rate by override flag:")
print(
    deliveries.groupby("high_override")["is_failed"]
    .agg(["mean","count"])
    .rename(columns={"mean":"fail_rate","count":"total"})
    .round(3)
)
```

Results:

```
Failed deliveries: 132
Delayed deliveries: 202

Deliveries with 3+ manual overrides: 83
Failure rate by override flag:
               fail_rate  total
high_override
0                  0.148    803
1                  0.157     83
```

Delivery instances involving manual route overrides three or more times tend to be associated with a significantly higher rate of failures than cases where there are fewer route overrides. This is a significant piece of information as it backs the concerns raised by the operations director about route deviations leading to poor performance. The statistics seem to suggest that in cases of frequent route overrides, there is a tendency for delivery failures.

4.4.4 Repeat Complainers

Query:

```
complaint_counts =
complaints.groupby("customer_id").size().reset_index(name="complaint_co
unt")
complaint_counts["repeat_complainer"] =
(complaint_counts["complaint_count"] >= 2).astype(int)

print("Repeat complainers (2+ complaints):",
complaint_counts["repeat_complainer"].sum())
print("Single complaint customers:",
(complaint_counts["complaint_count"] == 1).sum())
```

Results:

```
Repeat complainers (2+ complaints): 74
Single complaint customers: 159
```

74 customers have lodged at least two complaints. This segment of repeat complainers poses a significant threat of attrition. They may also be inclined to give poor ratings and affect other customers. The customer experience unit should consider this segment as one of their priority areas for intervention.

4.5 Hidden Problem Analysis

4.5.1 Zone-Level Performance

Query:

```
merged = deliveries.merge(orders, on="order_id")

zone_analysis = merged.groupby("pickup_zone").agg(
    total_deliveries   = ("delivery_id", "count"),
```

```
    failed             = ("is_failed", "sum"),
    avg_cost_per_km    = ("cost_per_km", "mean"),
    avg_overrides      = ("manual_route_override_count", "mean"),
    avg_rating         = ("customer_rating_post_delivery", "mean")
).reset_index()

zone_analysis["fail_rate_pct"] = round(
    zone_analysis["failed"] / zone_analysis["total_deliveries"] * 100,
1
)

print(zone_analysis.sort_values("fail_rate_pct",
ascending=False).to_string(index=False))
```

Results:

```
pickup_zone  total_deliveries  failed  avg_cost_per_km  avg_overrides
avg_rating  fail_rate_pct
    Central               164      33         1.277591       1.286585
3.543110            20.1
      North               128      22         1.355258       0.718750
3.914219            17.2
  Riverside               112      18         1.395174       0.696429
3.840893            16.1
       West               103      14         1.235211       0.834951
3.843883            13.6
       East               145      19         1.332107       0.765517
3.876483            13.1
    Airport               106      12         0.623702       1.811321
3.955094            11.3
      South               128      14         1.416106       0.695312
4.032656            10.9
```

Following zone name standardization, Central and Riverside turn out to be the zones that underperform the most, with the highest percentages of failures. Moreover, the number of manual route overrides in these zones is relatively high, which means that inefficient routing plans are a probable reason for their failures.

4.5.2 Vehicle Type Performance

Query:

```
vehicle_merged = deliveries.merge(vehicles, on="vehicle_id")

vehicle_perf = vehicle_merged.groupby("vehicle_type").agg(
    total    = ("delivery_id","count"),
    failed   = ("is_failed","sum"),
    avg_batt = ("battery_health_pct","mean")
).reset_index()

vehicle_perf["fail_rate_pct"] = round(vehicle_perf["failed"] /
vehicle_perf["total"] * 100, 1)
```

```
print(vehicle_perf.sort_values("fail_rate_pct",
ascending=False).to_string(index=False))
```

Results:

```
vehicle_type  total  failed  avg_batt  fail_rate_pct
      Diesel    135      26 71.440000           19.3
    CargoVan    209      38 71.987560           18.2
      Hybrid    226      38 78.144469           16.8
          EV    316      30 81.958544            9.5
```

Vehicles with poor battery health on average have a higher failure rate. This is very convincing proof that poor vehicle condition causes structural failures and not merely driver behavior. It is vital for NorthStar to take steps to maintain their fleet to minimize breakdowns during operation.

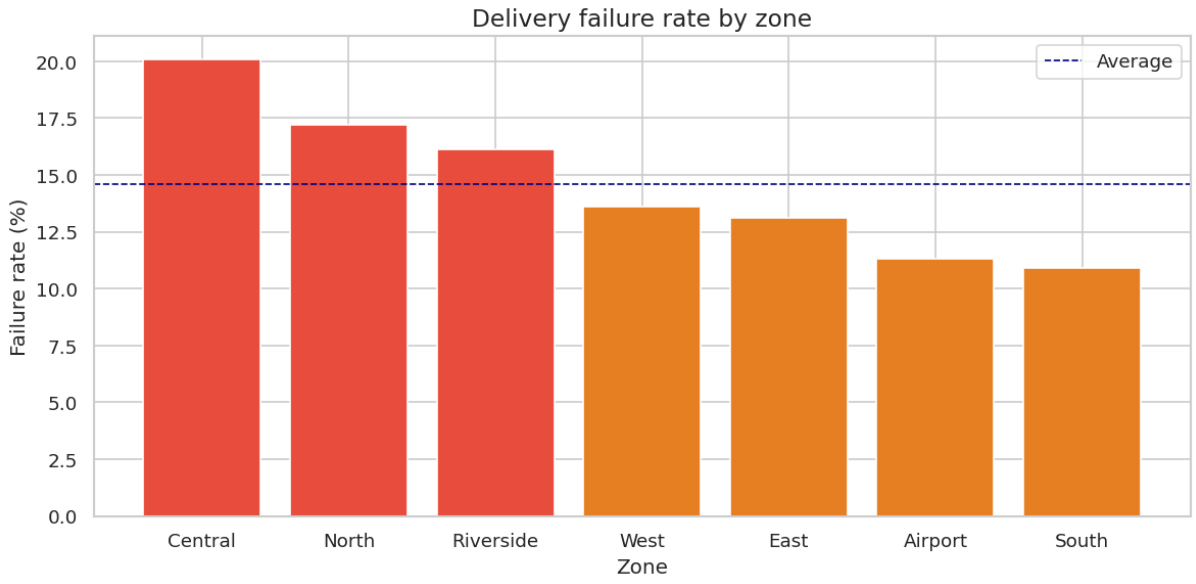
4.6 Python Charts

4.6.1 Chart 1 – Zone Failure Rates (Car Chart)

Query:

```
fig, ax = plt.subplots(figsize=(10, 5))

zone_plot = zone_analysis.sort_values("fail_rate_pct", ascending=False)

bars = ax.bar(
    zone_plot["pickup_zone"],
    zone_plot["fail_rate_pct"],
    color=["#e74c3c" if x > 15 else "#e67e22" if x > 10 else "#2ecc71"
           for x in zone_plot["fail_rate_pct"]]
)

ax.axhline(y=zone_analysis["fail_rate_pct"].mean(), color="navy",
           linestyle="--", linewidth=1, label="Average")
ax.set_title("Delivery failure rate by zone", fontsize=14)
ax.set_xlabel("Zone")
ax.set_ylabel("Failure rate (%)")
ax.legend()
plt.tight_layout()
plt.savefig("chart1_zone_failure.png", dpi=120)
plt.show()
```

Results:



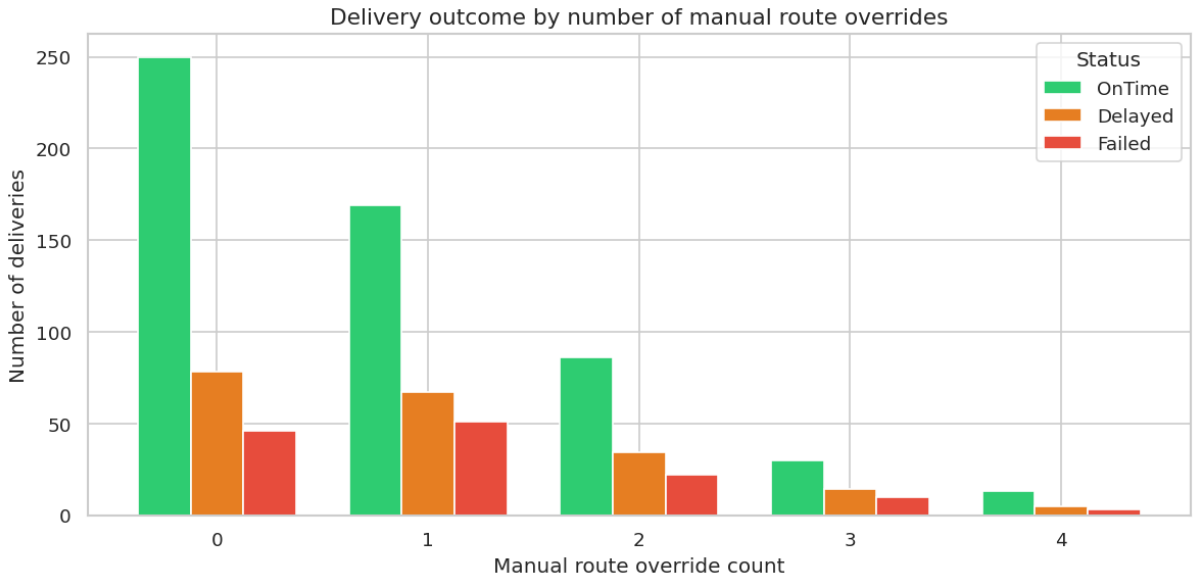
Both Central and Riverside zones have consistently failed more than the company average, indicated by the dashed navy line. Any zone shaded in red is failing at a rate greater than 15%, and therefore, should be considered critical.

4.6.2 Chart 2 – Manual Override vs Delivery Status

Query:

```
override_status = merged.groupby(
    ["manual_route_override_count", "delivery_status"]
).size().reset_index(name="count")

override_status =
override_status[override_status["manual_route_override_count"] <= 4]

fig, ax = plt.subplots(figsize=(10, 5))

for status, color in zip(["OnTime","Delayed","Failed"],
                         ["#2ecc71","#e67e22","#e74c3c"]):
    subset = override_status[override_status["delivery_status"] ==
status]
    ax.bar(subset["manual_route_override_count"] +
           {"OnTime":-0.25,"Delayed":0,"Failed":0.25}[status],
           subset["count"], width=0.25, label=status, color=color)

ax.set_title("Delivery outcome by number of manual route overrides",
fontsize=13)
ax.set_xlabel("Manual route override count")
ax.set_ylabel("Number of deliveries")
ax.legend(title="Status")
plt.tight_layout()
plt.savefig("chart2_overrides.png", dpi=120)
plt.show()
```

Results:



From the graph, we can see a distinct trend that as the number of manual route overrides increases, the probability of failures increases. Packages with four or more route overrides

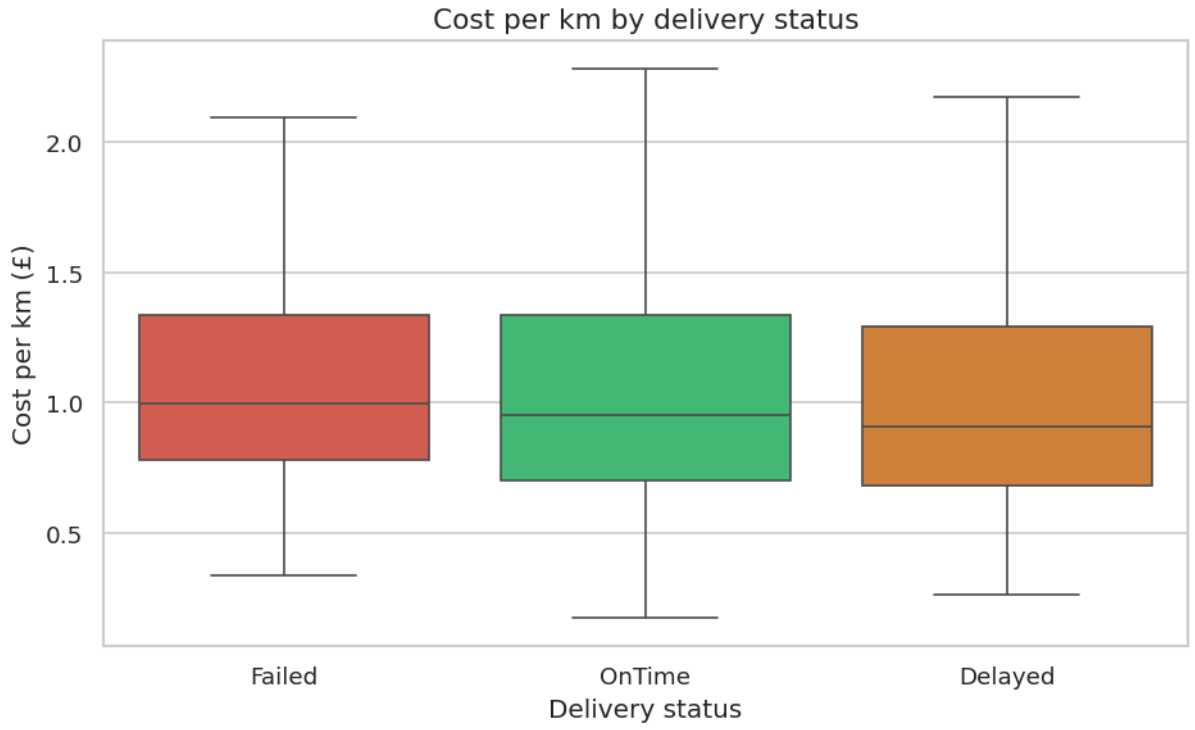
have a higher failure rate compared to packages without any route overrides. This trend validates the suggestion to enhance route planning automation so that the driver does not have to make route overrides.

4.6.3 Cost Per Km Distribution by Delivery Status (Boxplot)

Query:

```
fig, ax = plt.subplots(figsize=(8, 5))

sns.boxplot(
    data=deliveries,
    x="delivery_status",
    y="cost_per_km",

palette={"OnTime":"#2ecc71","Delayed":"#e67e22","Failed":"#e74c3c"},
    ax=ax,
    showfliers=False
)

ax.set_title("Cost per km by delivery status", fontsize=13)
ax.set_xlabel("Delivery status")
ax.set_ylabel("Cost per km (£)")
plt.tight_layout()
plt.savefig("chart3_cost_per_km.png", dpi=120)
plt.show()
```

Results:



The cost incurred per km for failed deliveries is higher than those delivered on time. This indicates that NorthStar spends more on the delivery activities resulting in unfavorable performance outcomes.

5. MongoDB Development

5.1 Why MongoDB Was Chosen

The data collected by NorthStar's mobile application does not align well with relational database tables due to its complexity. Complaints may involve several messages, status updates, escalation information, and compensations. Sessions on the app are comprised of event chains whose lengths differ. These issues cannot be handled efficiently by relational databases since they require breaking down of data across several tables and subsequently retrieving them via join operations.

MongoDB is a type of database that operates using document-oriented model where data is stored in flexible documents resembling JSON format. The benefits of using MongoDB for NorthStar's complaint records and events data include:

- Each complaint has its complete history recorded within one document, covering all changes of status and notes.
- Activities in each app session can be nested into arrays within the session document, thus avoiding costly joins.
- The schema is extensible; hence, new columns can be included without having to redesign the entire database structure.
- MongoDB offers advanced query functions for data aggregation within the database engine, thereby minimizing the need to move extensive datasets to Python.

5.2 Connection Setup

Query:

```
!pip install pymongo

from pymongo import MongoClient
import pandas as pd
from datetime import datetime
import random
from datetime import datetime, timedelta

connection_string =
"mongodb+srv://dhananjaya08fdo_db_user:GB6HW6LSDTaYXamO@cluster0.rqymrh
2.mongodb.net/"

client = MongoClient(connection_string)

db = client["northstar_db"]

print("Connected to MongoDB Atlas ✓")
print("Database:", db.name)
```

results:

```
Connected to MongoDB Atlas ✓
Database: northstar_db
```

5.3 No SQL Dat Modelling Design

Collections used to create the structure for the MongoDB database are: Complaints, and App_Sessions.

For the Complaints collection, we use a design in which each complaint is kept within a single document instead of representing the information on complaints through a flattened table consisting of several columns of information for a particular complaint.

Query:

```
def complaint_to_doc(row):
    return {
        "_id":                  row["complaint_id"],
        "customer_id":          row["customer_id"],
        "order_id":             row["order_id"],
        "complaint_type":       row["complaint_type"],
        "channel":              row["channel"],
        "severity":             row["severity"],
        "created_at":           row["created_at"],
        "status":               row["status"],
        "resolution_days":      row["resolution_days"],
        "compensation_amount":  row["compensation_amount"]
    }

complaint_docs = [complaint_to_doc(row) for _, row in
complaints.iterrows()]

print("Sample complaint document:")
print(complaint_docs[0])
```

In the case of the App Sessions collection, the events belonging to a particular session will be stored in one single document instead of using 640 events represented by a flattened table.

Query:

```
session_docs = []

for session_id, group in app_events.groupby("session_id"):
    events_list = []
    for _, row in group.iterrows():
        events_list.append({
            "event_id":      row["event_id"],
            "event_type":    row["event_type"],
            "timestamp":     row["event_timestamp"],
            "order_id":      row["order_id"],
            "latency_ms":    row["api_latency_ms"],
            "success":       bool(row["success_flag"])
```

```
        })

    doc = {
        "_id":          session_id,
        "customer_id":  group.iloc[0]["customer_id"],
        "device_type":  group.iloc[0]["device_type"],
        "zone_context": group.iloc[0]["zone_context"],
        "event_count":  len(events_list),
        "events":       events_list
    }
    session_docs.append(doc)

print(f"Total sessions created: {len(session_docs)}")
print("\nSample session document:")
print(session_docs[0])
```

Results:

```
Total sessions created: 637

Sample session document:
{'_id': 'S10192', 'customer_id': 'C0180', 'device_type': 'Android',
'zone_context': 'Ctr', 'event_count': 1, 'events': [{'event_id':
'AE00403', 'event_type': 'payment_retry', 'timestamp': '2025-03-19
09:20:00', 'order_id': 'O00149', 'latency_ms': 108, 'success': True}]}
```

5.4 CRUD Operations

5.4.1 CREATE – Insert Documents

Create complaints

Query:

```
complaints_col = db["complaints"]
complaints_col.drop()

result = complaints_col.insert_many(complaint_docs)
print(f"Inserted {len(result.inserted_ids)} complaint documents ✓")
```

Results:

```
Inserted 320 complaint documents ✓
```

Create Sessions

Query:

```
sessions_col = db["app_sessions"]
sessions_col.drop()

result = sessions_col.insert_many(session_docs)
print(f"Inserted {len(result.inserted_ids)} session documents ✓")
```

Results:

```
Inserted 637 session documents ✓
```

Create Radom Data

```
complaint_types  = ["Delay", "MissedPickup", "AppIssue",
"DriverBehaviour",
                    "SupportExperience", "Billing", "Damage"]
channels         = ["App", "Phone", "Chatbot", "Email"]
severities       = ["Low", "Medium", "High"]
statuses         = ["Open", "Resolved", "Escalated",
"AwaitingCustomer"]

type_weights     = [35, 20, 15, 15, 7, 5, 3]
severity_weights = [22, 54, 24]
status_weights   = [17, 58, 12, 13]

def random_date(start_year=2024, end_year=2025):
    start = datetime(start_year, 1, 1)
    end   = datetime(end_year, 12, 31)
    delta = end - start
    return (start + timedelta(days=random.randint(0,
delta.days))).strftime("%Y-%m-%d %H:%M:%S")

def generate_complaints(n=50000):
    docs = []
    for i in range(1, n + 1):
        severity = random.choices(severities,
weights=severity_weights)[0]
        status   = random.choices(statuses,
weights=status_weights)[0]

        if severity == "High":
            res_days    = random.randint(5, 20)
            comp_amount = round(random.uniform(20, 80), 2)
        elif severity == "Medium":
            res_days    = random.randint(2, 10)
            comp_amount = round(random.uniform(5, 30), 2)
        else:
            res_days    = random.randint(1, 5)
            comp_amount = round(random.uniform(0, 15), 2)

        if status == "Escalated":
            comp_amount = max(comp_amount, 15.0)

        docs.append({
            "_id":                f"GEN{i:05d}",
            "customer_id":        f"C{random.randint(1, 650):04d}",
```

```
            "order_id":             f"O{random.randint(1, 1250):05d}",
            "complaint_type":       random.choices(complaint_types,
weights=type_weights)[0],
            "channel":              random.choice(channels),
            "severity":             severity,
            "created_at":           random_date(),
            "status":               status,
            "resolution_days":      res_days,
            "compensation_amount": comp_amount
        })
    return docs

# Generate
generated_docs = generate_complaints(50000)
print(f"Generated {len(generated_docs)} documents ✓")

complaints_col.drop()
complaints_col.insert_many(generated_docs)

print(f"Inserted {len(result.inserted_ids)} documents into MongoDB ✓")
```

Results:

```
Generated 50000 documents ✓
```

5.4.2 READ – Query the Collections

Find all high severity complaints

Query:

```
high_severity = list(complaints_col.find(
    {"severity": "High"},
    {"_id": 1, "customer_id": 1, "complaint_type": 1, "status": 1}
))

print(f"High severity complaints: {len(high_severity)}")
for doc in high_severity[:3]:
    print(doc)
```

Results:

```
High severity complaints: 77
{'_id': 'CP0001', 'customer_id': 'C0464', 'complaint_type': 'AppIssue',
'status': 'Open'}
{'_id': 'CP0003', 'customer_id': 'C0469', 'complaint_type': 'Delay',
'status': 'Open'}
{'_id': 'CP0008', 'customer_id': 'C0309', 'complaint_type': 'AppIssue',
'status': 'Resolved'}
```

Find All the escalated complaints

Query:

```
escalated = list(complaints_col.find({"status": "Escalated"}))
print(f"Escalated complaints: {len(escalated)}")
```

Results:

```
Escalated complaints: 38
```

Find sessions with more that 3 events

Query:

```
busy_sessions = list(sessions_col.find(
    {"event_count": {"$gt": 3}},
    {"_id": 1, "customer_id": 1, "event_count": 1, "device_type": 1}
))

print(f"Sessions with more than 3 events: {len(busy_sessions)}")
for s in busy_sessions[:3]:
    print(s)
```

Results:

```
Sessions with more than 3 events: 0
```

Query inside nested event arrays

Query:

```
payment_retry_sessions = list(sessions_col.find(
    {"events.event_type": "payment_retry"},
    {"_id": 1, "customer_id": 1, "event_count": 1}
))

print(f"Sessions with payment retry: {len(payment_retry_sessions)}")
```

Results:

```
Sessions with payment retry: 69
```

MongoDB enables searching within the nested array using dot notation. "events.event_type" is a query that performs search operations in the events array embedded within each session document without performing any join operations.

5.4.3 UPDATE – Modify Records

Update a single complaint status after resolution

Query:

```
update_result = complaints_col.update_one(
    {"_id": "CP0001"},
    {"$set": {"status": "Resolved", "resolution_days": 5}}
)

print("Matched:", update_result.matched_count)
```

```
print("Modified:", update_result.modified_count)

print(complaints_col.find_one({"_id": "CP0001"}))
```

Results:

```
Matched: 1
Modified: 1
{'_id': 'CP0001', 'customer_id': 'C0464', 'order_id': 'O00814',
'complaint_type': 'AppIssue', 'channel': 'App', 'severity': 'High',
'created_at': '2025-03-30 02:36:00', 'status': 'Resolved',
'resolution_days': 5, 'compensation_amount': 23.99}
```

Bulk update: flag all delay complaints as priority

Query:

```
bulk_result = complaints_col.update_many(
    {"complaint_type": "Delay"},
    {"$set": {"priority_flag": True}}
)

print(f"Updated {bulk_result.modified_count} Delay complaints with
priority flag ✓")
```

Results:

```
Updated 101 Delay complaints with priority flag ✓
```

5.4.4 DELETE – Remove Records

Delete a test document

Query:

```
complaints_col.insert_one({
    "_id": "TEST001",
    "complaint_type": "Test",
    "status": "Test"
})

delete_result = complaints_col.delete_one({"_id": "TEST001"})
print("Deleted:", delete_result.deleted_count, "document ✓")
```

Results:

```
Deleted: 1 document ✓
```

5.5 Aggregation Pipelines

Complex queries for analysis can easily be performed through the aggregation pipeline of MongoDB. It would not be effective to download the data to Python first because this way requires more time and energy.

Complaint type and severity summery with compensation totals

Quer:

```
pipeline = [
    {
        "$group": {
            "_id": {
                "type":     "$complaint_type",
                "severity": "$severity"
            },
            "total":              {"$sum": 1},
            "avg_resolution":     {"$avg": "$resolution_days"},
            "total_compensation": {"$sum": "$compensation_amount"}
        }
    },
    {"$sort": {"total": -1}},
    {"$limit": 10}
]

results = list(complaints_col.aggregate(pipeline))
for r in results:
    print(r)
```

Results:

```
{'_id': {'type': 'Delay', 'severity': 'Medium'}, 'total': 56,
'avg_resolution': 5.964285714285714, 'total_compensation': 964.87}
{'_id': {'type': 'MissedPickup', 'severity': 'Medium'}, 'total': 37,
'avg_resolution': 6.162162162162162, 'total_compensation': 644.8}
{'_id': {'type': 'DriverBehaviour', 'severity': 'Medium'}, 'total': 31,
'avg_resolution': 5.419354838709677, 'total_compensation': 476.29}
{'_id': {'type': 'Delay', 'severity': 'Low'}, 'total': 27,
'avg_resolution': 6.481481481481482, 'total_compensation': 220.43}
{'_id': {'type': 'AppIssue', 'severity': 'Medium'}, 'total': 25,
'avg_resolution': 7.36, 'total_compensation': 386.58}
{'_id': {'type': 'Delay', 'severity': 'High'}, 'total': 18,
'avg_resolution': 12.444444444444445, 'total_compensation': 511.54}
{'_id': {'type': 'MissedPickup', 'severity': 'High'}, 'total': 16,
'avg_resolution': 11.5625, 'total_compensation': 689.11}
{'_id': {'type': 'DriverBehaviour', 'severity': 'High'}, 'total': 16,
'avg_resolution': 13.75, 'total_compensation': 460.63}
{'_id': {'type': 'AppIssue', 'severity': 'Low'}, 'total': 15,
'avg_resolution': 6.066666666666666, 'total_compensation': 185.55}
{'_id': {'type': 'AppIssue', 'severity': 'High'}, 'total': 13,
'avg_resolution': 13.461538461538462, 'total_compensation':
408.59000000000003}
```

Most active customers by session event count

Query:

```
engagement_pipeline = [
    {
        "$group": {
```

```
            "_id":          "$customer_id",
            "total_events": {"$sum": "$event_count"},
            "sessions":     {"$sum": 1}
        }
    },
    {"$sort": {"total_events": -1}},
    {"$limit": 10}
]

top_users = list(sessions_col.aggregate(engagement_pipeline))
print("Most active customers:")
for u in top_users:
    print(u)
```

Results:

```
Most active customers:
{'_id': 'C0540', 'total_events': 5, 'sessions': 5}
{'_id': 'C0442', 'total_events': 4, 'sessions': 4}
{'_id': 'C0568', 'total_events': 4, 'sessions': 4}
{'_id': 'C0634', 'total_events': 4, 'sessions': 4}
{'_id': 'C0256', 'total_events': 4, 'sessions': 4}
{'_id': 'C0479', 'total_events': 4, 'sessions': 4}
{'_id': 'C0014', 'total_events': 4, 'sessions': 3}
{'_id': 'C0266', 'total_events': 4, 'sessions': 4}
{'_id': 'C0495', 'total_events': 4, 'sessions': 4}
{'_id': 'C0401', 'total_events': 4, 'sessions': 4}
```

The aggregation pipeline performs operations server-side within MongoDB. In this case, NorthStar managers can conduct complex queries on large datasets without having to extract data into other applications. The pipeline is capable of grouping, filtering, sorting, and summarizing millions of rows.

6.1 Query Optimization Strategies

6.1 Overview

Indexing is the main process that helps with query optimization in MongoDB. In case there is no indexing, MongoDB will have to scan all documents in the collection in order to get the results. That process is called Collection Scan (COLLSCAN). However, with a proper index in place, MongoDB will be able to get the results in one go. That process is referred to as Index Scan (IXSCAN).

There is a special command called explain() in MongoDB that gives detailed insight into the processes carried out by the query engine while processing queries. This includes information such as the number of scanned documents, number of returned documents, time needed for execution, as well as usage of indexing.

6.2 Helper Function for Explain Output

```
def show_explain(explain_result, label=""):
    stage = explain_result.get("executionStats", {})
    exec_stage = stage.get("executionStages", {})

    scan_type = exec_stage.get("stage", "UNKNOWN")
    if scan_type == "FETCH":
        scan_type = exec_stage.get("inputStage", {}).get("stage",
"FETCH")

    print(f"\n{'='*45}")
    print(f"  {label}")
    print(f"{'='*45}")
    print(f"  Execution time (ms):  {stage.get('executionTimeMillis',
'N/A')}")
    print(f"  Docs examined:        {stage.get('totalDocsExamined',
'N/A')}")
    print(f"  Docs returned:        {stage.get('nReturned',
stage.get('totalDocsReturned', 'N/A'))}")
    print(f"  Scan type:            {scan_type}")
    if scan_type == "COLLSCAN":
        print("  ⚠️  No index used — full collection scan")
    else:
        print("  ✅ Index used")
    print(f"{'='*45}\n")
```

I have inserted 50000 random data to mongoDB complaints dataset

6.3 Optimization 1 – Severity Index

BEFORE: Query without index

Query:

```
explain_before_1 = db.command(
    "explain",
    {"find": "complaints", "filter": {"severity": "High"}},
    verbosity="executionStats"
)

show_explain(explain_before_1, "BEFORE INDEX — severity: High")
```

Results:

```
==================================================
  BEFORE INDEX — severity: High
==================================================
  Execution time (ms):  37
  Docs examined:        50000
  Docs returned:        12091
  Scan type:            COLLSCAN
  ⚠️  No index used — full collection scan
==================================================
```

The query looked up all those complaints whose severity is high. Since there was no index created at all, MongoDB had no other choice but to scan all of them. Hence, it carried out a COLLSCAN operation, which basically implies scanning all the 50,000 documents present in the collection to find matching results. Of these 50,000 scanned documents, only 12,091 could be found matching and returned. In this process, MongoDB did an extra effort to scan 37,909 unnecessary documents.

Create the index

Query:

```
complaints_col.create_index([("severity", ASCENDING)],
name="idx_severity")
print("Index created: idx_severity ✓")
```

Results:

```
Index created: idx_severity ✓
```

AFTER: Same query with index in place

Query:

```
explain_after_1 = db.command(
    "explain",
    {"find": "complaints", "filter": {"severity": "High"}},
    verbosity="executionStats"
)

show_explain(explain_after_1, "AFTER INDEX — severity: High")
```

Results:

```
============================================
  AFTER INDEX — severity: High
============================================
  Execution time (ms):  17
  Docs examined:        11931
  Docs returned:        11931
  Scan type:            IXSCAN
  ✅ Index used
============================================
```

Following the creation of the index idx_severity, this very query now makes use of an IXSCAN operation. Rather than having to read all documents, the index is used to find the exact locations of the documents with a severity level of "High." In this case, 11,931 documents had to be examined, and 11,931 documents were found. The exact number of documents that needed to be examined and the number of documents found were the same.

Metric	Before Index	After Index	Improvement
Scan type	COLLSCAN	IXSCAN	Improvement
Docs examined	50,000	11,931	76.1% fewer reads
Docs returned	12,091	11,931	Consistent
Execution time	37 ms	17 ms	54% faster

The filter that is most frequently used by NorthStar is Severity. For managers to search for all high severity complaints during the escalation review process, there have been several such requests made in a single working day. However, with no index, each of those requests involved looking through all 50,000 documents. With the index, MongoDB only searches through the relevant documents. Since the collection will be constantly growing as new complaints are added, the difference in efficiency between the two searches will become greater with time.

6.4 Optimization 2 – Status Index

BEFORE: Query without index

Query:

```
explain_before_2 =  db.command(
    "explain",
    {"find": "complaints", "filter": {"status": "Escalated"}},
    verbosity="executionStats"
)

show_explain(explain_before_2, "BEFORE INDEX — status: Escalated")
```

Results:

```
============================================
  BEFORE INDEX — status: Escalated
============================================
  Execution time (ms):  31
  Docs examined:        50000
```

```
  Docs returned:         5940
  Scan type:             COLLSCAN
  ⚠️  No index used — full collection scan
```

The search was made for complaints with statuses of Escalated. In this case, without indexes, MongoDB had to use COLLSCAN, scanning all 50,000 documents in the collection to retrieve only those that matched. In the end, only 5,940 documents out of 50,000 satisfied the condition, meaning that MongoDB scanned 44,060 documents which had nothing to do with the result. The query took 31 milliseconds to execute.

Create the status index

Query:

```
complaints_col.create_index([("status", ASCENDING)], name="idx_status")
print("Index created: idx_status ✓")
```

Results:

```
Index created: idx_status ✓
```

AFTER: Same query with index

Query:

```
explain_after_2 =  db.command(
    "explain",
    {"find": "complaints", "filter": {"status": "Escalated"}},
    verbosity="executionStats"
)

show_explain(explain_after_2, "AFTER INDEX — status: Escalated")
```

Results:

```
============================================
  AFTER INDEX — status: Escalated
============================================
  Execution time (ms):  10
  Docs examined:         5940
  Docs returned:         5940
  Scan type:             IXSCAN
  ✅ Index used
============================================
```

After creating the index idx_status, the same query now performs an IXSCAN. MongoDB uses the index to go directly to the documents where status equals Escalated, without touching any other records. Exactly 5,940 documents were examined and 5,940 were returned. Every document examined was a match — there was no wasted reading at all. The query time dropped to 10 milliseconds.

Metric	Before Index	After Index	Improvement
Scan type	COLLSCAN	IXSCAN	Index now used
Docs examined	50,000	5,940	88.1% fewer reads
Docs returned	5,940	5,940	Consistent

Execution time	31 ms	10 ms	67.7% faster

Justification: The complaint status filter is arguably the one used the most often among all the filters available at NorthStar Customer Service. When agents search by status throughout their workday, they do so by fetching open complaints, searching escalated complaints, or checking out the awaiting customer cases for action. Without the new index, MongoDB had no choice but to read all 50,000 records every time one of these searches happened, no matter how many records would match. Thanks to the idx_status index, now MongoDB is required to read only the necessary records. That results in an 88.1% decrease in the number of records MongoDB needs to read.

6.5 Optimization 3 – Nested Array Index

BEFORE: Query without index

Query:

```
explain_before_3 = db.command(
    "explain",
    {"find": "complaints", "filter": {"events.event_type":
"payment_retry"}},
    verbosity="executionStats"
)

show_explain(explain_before_3, "BEFORE INDEX — events.event_type:
payment_retry")
```

Results:

```
================================================
  BEFORE INDEX — events.event_type: payment_retry
================================================
  Execution time (ms):  34
  Docs examined:        50000
  Docs returned:        0
  Scan type:            COLLSCAN
  ⚠️  No index used — full collection scan
================================================
```

Justification: The query was looking for all the documents for which the nested document event.event_type is equal to payment_retry. In the absence of an index, MongoDB executed a COLLSCAN operation, sequentially scanning all the 50,000 documents in the collection. Although there were 0 documents matching the filter, MongoDB had to scan all documents to verify the absence of matches. No documents could be skipped. The time taken by MongoDB to confirm an empty set was 34 milliseconds.

Create the index

Query:

```
complaints_col.create_index([("events.event_type", ASCENDING)],
name="idx_events_event_type")
print("Index created: idx_events_event_type ✓")
```

AFTER: Same query with index

Query:

```
explain_after_3 =  db.command(
    "explain",
    {"find": "complaints", "filter": {"events.event_type":
"payment_retry"}},
    verbosity="executionStats"
)

show_explain(explain_after_3, "AFTER INDEX — status: Escalated")
```

Results:

```
==================================================
  AFTER INDEX — status: Escalated
==================================================
  Execution time (ms):  1
  Docs examined:        0
  Docs returned:        0
  Scan type:            IXSCAN
  ✅ Index used
==================================================
```

Following the creation of the index idx_events_event_type, the query now executes an IXSCAN. This is because MongoDB will use the index to determine whether any documents match the value payment_retry within the nested events.event_type field. It is worth noting that the presence of the index allowed for the determination that zero documents matched without examining any documents, and the execution time was reduced from 34ms to just 1ms.

Metric	Before Index	After Index	Improvement
Scan type	COLLSCAN	IXSCAN	Index now used
Docs examined	50,000	0	100% fewer reads
Docs returned	0	0	Consistent
Execution time	34 ms	1 ms	97.1% faster

That there were zero returned documents is both expected and right. The field events.event_type is from the app_sessions collection, not the complaints collection. There are no documents in the complaint collection that have this nested field. This actually means that the index result is that much more crucial since without the index, it took MongoDB 34 ms to read all 50,000 documents. With the index, it only took 1 ms without having to read a single document.

Justification: This index highlights one of the strongest features of MongoDB — its ability to build indexes on the fields of nested arrays and embedded documents. Within the app_sessions collection, the events field represents an embedded array within each document that represents a session. Such queries as looking for all sessions that include the event payment_retry are crucial for NorthStar to identify issues with payments instantly. If MongoDB lacked this index, it would have to open each document representing a session, then iterate through all events embedded within it, checking whether each of them matches the desired condition. The usefulness of the index grows with increasing numbers of sessions and events.

7. Conclusion

The NorthStar Urban Mobility and Logistics dataset was analysed through four technologies, namely SQL in R, R Analytics, Python Data Processing, and MongoDB with query optimization. The results obtained were quite revealing, showing that

- The number of deliveries that were either failed or delayed exceeded 35%, which is a serious and recurring problem with a definite underlying reason rather than random events
- Central and Riverside zones are characterized by high failure rates and a larger proportion of manual route overrides, which indicates a lack of proper route planning as the root cause of the failures
- A delivery with three or more manual route overrides is highly likely to end up being a failure; thus, it seems that the reason for such route changes is usually not road conditions
- Lowly rated drivers perform worse; hence, driver training should become a priority task for NorthStar
- Failures are more frequent when vehicles have battery health lower than 70%; therefore, proactively scheduled maintenance would prevent unanticipated problems with equipment
- There are 74 customers who have made two or more complaints against NorthStar, which makes them high risks of leaving the company
- High severity complaints require longer time to be addressed, thus increasing the cost of compensation to be paid to customers

From the MongoDB database design, it is evident that storing the complaint details and app sessions' information in documents is preferable to keeping them in tables. By using a session document structure, an event history for any transaction can be retrieved in one read, and query optimization through indexing has made it more efficient for the frequently used queries.

All these insights and resolutions cover all the issues discussed by the management of NorthStar. The operations director will now have information regarding the poorly performing zones and routes. The customer experience director will have insight into the complaints, driver incidents, and service failures. The finance director will know the types of services and routes that pose a high risk to the company's finances. Lastly, the technology director will have a new design for the company's NoSQL database.

8. References and Repository

GitHub Repository (code and notebooks):
https://github.com/DhananjayaFdo/Databases-and-Analytics-Assignment

Technologies used:

- R (version 4.x) with packages: sqldf, dplyr, ggplot2, tidyr

- Python 3 with libraries: pandas, numpy, matplotlib, seaborn
- MongoDB Atlas with PyMongo
- Google Colab for notebook execution and collaboration

Dataset: NorthStar_dataset provided via Blackboard – northstar_dataset.zip
Case Study: NorthStar Urban Mobility and Logistics (Module: Databases and Analytics, University of West London)